NexoraDB راهنمای نصب، اجرا، تست و ارائه

سناریوی یکپارچه NexoraQL: Document Store + Live Graph + الگوریتم ۱۲

هدف این راهنما

این سند برای اجرای سریع پروژه و یک دمو منسجم در ارائه آماده شده است .سناریو یک شبکهٔ اجتماعی کوچک می‌سازد : کاربران و پست‌ها به‌صورت سند ذخیره می‌شوند، رابطهٔ دنبال‌کردن به گراف زنده نگاشت می‌شود و در پایان هر ۱۲ الگوریتمِ پیاده‌سازی‌شده اجرا می‌شوند.

ترتیب پیشنهادی ارائه

نصب و بالا آوردن پنل ۲ (ساخت داده‌های سندی ۳ (ساخت و پیمایش گراف ۴ (اجرای تحلیل‌های سریع ۵ (اجرای ۱) تحلیل‌های سنگینِ پس‌زمینه

را بزنید و شناسهٔ بازگشتی را SHOW JOBS ON social ابتدا RUN JOB در اجرای واقعی پویا است؛ پس بعد از Job نکته :نام جای‌گذاری کنید JOB STATUS / JOB RESULT در.

۱. نصب و آماده‌سازی

روش پیشنهادی برای اجرای پروژه (آماده **wheel** باینری)

```
cd NexoraDB
python۳,۱۰ -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install dist/nexoradb-۰,۱,۰-cp۳۱۰-cp۳۱۰-manylinux_۲_۳۵_x۸۶_٦٤.whl
python -m pip install -e '.[dev]'
python -m pip install Faker
```

این wheel در وضعیت فعلی برای Ubuntu x86_64 و CPython 3.10 برای توسعهٔ صرفِ parser آماده شده است. و editable نیز کافی است. تست‌ها، نصب

ساخت هستهٔ C++ (در صورت نیاز) از سورس

```
export VCPKG_ROOT=/path/to/vcpkg
mkdir -p build && cd build
cmake .. -DNEXORA_BUILD_GRAPH=ON -DNEXORA_BUILD_PYTHON=ON -DCMAKE_BUILD_TYPE=Release
cmake --build . -j$(nproc)
```

۲. اجرا

```
# ۱ ترمینال: API
nexoradb server --host ۰,۰,۰,۰ --port ۸۰۰۰


# ۲ ترمینال داشبورد: React
nexoradb dashboard --host ۱۲۷,۰,۰,۱ --port ٥۱۷۳


# یا هر دو برای توسعه
nexoradb dev


# پنل آمادهٔ داخل بسته نیز روی این آدرس در دسترس است
# http://localhost:۸۰۰۰/admin


# ۳ ترمینال NexoraQL محیط تعاملی:
nexoradb cli --url http://localhost:۸۰۰۰
```

۳. تست

```
# از پوشهٔ NexoraDB
python -m pytest tests -q


# پوشش دقیق parser و ۱۲ الگوریتم
python -m pytest tests/test_parser_algorithms.py -q


# سناریوهای end-to-end زبان NexoraQL با fake engine
python -m pytest tests/test_parser_queries.py -q


# بنچمارک اجتماعی (نیازمند API و token با scope query:execute)
export NEXORADB_TOKEN='<app-token-with-query:execute-scope>'
```

```
python examples/social_media_benchmark.py
```

دمو :بخش سندی ۴.

ایجاد دادهٔ اولیه — A گام

```
-- A. داده‌های سندی :کاربران، پست‌ها و رابطهٔ دنبال‌کردن
CREATE COLLECTION users (_id STRING REQUIRED UNIQUE, username STRING REQUIRED UNIQUE, age
INT۱۶, active BOOL, city STRING) STRICT;
CREATE COLLECTION posts;
CREATE COLLECTION follows;


INSERT INTO users BATCH VALUES
  ('{"_id":"u۱","username":"ali","age":۲۸,"active":true,"city":"Tehran"}'),
  ('{"_id":"u۲","username":"sara","age":۲۵,"active":true,"city":"Shiraz"}'),
  ('{"_id":"u۳","username":"reza","age":۳۲,"active":true,"city":"Tehran"}'),
  ('{"_id":"u٤","username":"mina","age":۲۹,"active":true,"city":"Tabriz"}'),
  ('{"_id":"u٥","username":"navid","age":۲۷,"active":true,"city":"Tehran"}'),
  ('{"_id":"u٦","username":"roya","age":۳۰,"active":true,"city":"Mashhad"}');


INSERT INTO posts BATCH VALUES
  ('{"_id":"p۱","title":"Hello
NexoraDB","author_id":"u۱","likes":٥,"tags":["db","graph"]}'),
  ('{"_id":"p۲","title":"Graph Analytics","author_id":"u۲","likes":۱۲,"tags":["graph"]}');


INSERT INTO follows BATCH VALUES
  ('{"_id":"f۱","from_id":"u۱","to_id":"u۲","since":۱۷۰۰۰۰۰۰۰۰}'),
  ('{"_id":"f۲","from_id":"u۱","to_id":"u۳","since":۱۷۰۰۰۰۰۱۰۰}'),
  ('{"_id":"f۳","from_id":"u۲","to_id":"u۳","since":۱۷۰۰۰۰۰۲۰۰}'),
  ('{"_id":"f٤","from_id":"u۲","to_id":"u٤","since":۱۷۰۰۰۰۰۳۰۰}'),
  ('{"_id":"f٥","from_id":"u۳","to_id":"u٤","since":۱۷۰۰۰۰۰٤۰۰}'),
  ('{"_id":"f٦","from_id":"u٤","to_id":"u٥","since":۱۷۰۰۰۰۰٥۰۰}'),
  ('{"_id":"f۷","from_id":"u٥","to_id":"u۱","since":۱۷۰۰۰۰۰٦۰۰}'),
  ('{"_id":"f۸","from_id":"u٦","to_id":"u۲","since":۱۷۰۰۰۰۰۷۰۰}');
```

Document Store نمایش قابلیت‌های — B گام

```
CREATE UNIQUE INDEX idx_users_username ON users (username);
CREATE INDEX idx_posts_author ON posts (author_id);


SELECT _id, username, city FROM users WHERE age >= ۲۷ AND active = true LIMIT ۱۰;
SELECT * FROM posts LOOKUP JOIN users ON posts.author_id = users._id LIMIT ۱۰;
COUNT FROM users WHERE city = 'Tehran';
EXISTS IN users WHERE username = 'ali';


UPDATE users SET city='Karaj', updated_at=NOW() INCREMENT age BY ۱ ADD TO SET
badges='presenter' WHERE _id='u۱';
BEGIN TRANSACTION;
INSERT INTO posts VALUES ('{"_id":"p۳","title":"Transactional
post","author_id":"u۱","likes":۰}');
COMMIT;
```

دمو :ساخت و بررسی گراف زنده ۵.

```
CREATE LIVE GRAPH social HETEROGENEOUS DIRECTED;
USE GRAPH social;
MAP NODE User FROM users KEY _id PROPERTIES username, age, city;
MAP EDGE FOLLOWS FROM follows SOURCE from_id AS User TARGET to_id AS User DIRECTED
PROPERTIES since;
BUILD GRAPH social VERBOSE;


GRAPH STATUS social;
GRAPH STATS social;
RENDER GRAPH social;


TRAVERSE User('u۱') OUT FOLLOWS DEPTH ۲ LIMIT ۲۰;
GET NODE User('u۱');
EDGE EXISTS User('u۱') -[FOLLOWS]-> User('u۲');
```

JSON را اجرا کنید تا پیوند بین دادهٔ TRAVERSE را باز کنید .سپس Graphs پنل ،BUILD GRAPH نکتهٔ ارائه :بعد از اجرای .و پیمایش گراف کاملاً ملموس شود

(RUN LOCK) دمو :۷ الگوریتم سریع ۶.

.و برای پاسخ سریع به درخواست‌های موضعی اجرا می‌شوند LiveGraph این الگوریتم‌ها روی

```
-- ۱. دوستان مشترک
RUN LOCK MutualFriends ON social WITH user۱='u۱', user۲='u۲', edge_type='FOLLOWS';


-- ۲. دوستان یک کاربر /دریافت همسایه‌ها
RUN LOCK GetFriends ON social WITH user='u۱', edge_type='FOLLOWS' LIMIT ۲۰;


-- ۳. آیا دو کاربر با یک یال متصل‌اند؟
RUN LOCK AreConnected ON social WITH user۱='u۱', user۲='u۲', edge_type='FOLLOWS';


-- ٤. کوتاه‌ترین مسیر
RUN LOCK ShortestPath ON social WITH from='u۱', to='u٥', edge_type='FOLLOWS';


-- ٥. پیشنهاد دوستی
RUN LOCK FriendSuggestion ON social WITH user='u۱', edge_type='FOLLOWS' LIMIT ۱۰;


-- ٦. گره‌های پراتصال
RUN LOCK MostConnected ON social WITH metric='out', node_type='User' LIMIT ۳;


-- ۷. آمار شبکه
RUN LOCK NetworkStats ON social WITH mode='full';
```

۷. دمو ۵: الگوریتم سنگین (RUN JOB)

شناسهٔ SHOW JOBS ON social اجرا می‌شوند؛ بعد از هر اجرا از job گراف به‌صورت snapshot این‌ها روی JOB RESULT را بردارید و برای نتیجه از job استفاده کنید.

```
-- ۸. مؤلفه‌های هم‌بند
RUN JOB ConnectedComponents ON social WITH node_type='User';


-- ۹. تشخیص کامیونیتی با Label Propagation
RUN JOB CommunityDetection ON social WITH max_iterations=۱۰, min_community_size=۲,
members=true, node_type='User';


-- ۱۰. فاصلهٔ همهٔ گره‌ها از u۱
RUN JOB AllDistances ON social WITH source='u۱', all=true, max_hops=٤, node_type='User';


-- ۱۱. مرکزیت بینابینی
RUN JOB BetweennessCentrality ON social RETURNS TOP ٥;


-- ۱۲. بیشینه‌سازی نفوذ
RUN JOB InfluenceMaximization ON social WITH k=۳, simulations=۲٥, probability=۰٫۲۰;


-- کنترل jobها (خروجی از SHOW JOBS را بردارید / شناسه را)
SHOW JOBS ON social;
JOB STATUS 'job_id_from_output';
JOB RESULT 'job_id_from_output';
```

در PageRank و Neighborhood متصل و تست شده‌اند. نام‌های pybind نکتهٔ فنی: در نسخهٔ فعلی، ۱۲ الگوریتم بالا به parser دیده می‌شوند، اما جزو دوازده الگوریتمِ پیاده‌سازی‌شده و قابل‌دموی این نسخه نیستند مشخصات.

چک‌لیست اجرای ارائه

- پیش از ارائه: API، dashboard و CLI را باز کنید و token لازم برای benchmark را آماده داشته باشید.
- ابتدا بخش سندی را اجرا کنید و خروجی SELECT / LOOKUP JOIN را نشان دهید.
- سپس BUILD GRAPH، GRAPH STATS و TRAVERSE را باز کنید و نمای گراف را اجرا کنید.
- دستور ۷ RUN LOCK ۵ را پشت‌سرهم اجرا کنید؛ سپس RUN JOB را نشان دهید و وضعیت/نتیجه را ارسال کنید.
- در پایان: SNAPSHOT بودن تحلیل‌های سنگین و زنده‌بودن projection گراف از روی document ها را جمع‌بندی کنید.